

Building a Crypto Perpetual Futures Exchange From Scratch — Resource List

Language-agnostic. Every link below was checked and was live on 2026-09-26. Order = the order you should actually learn/build in.

A perp exchange = **spot exchange (matching engine) + margin accounting + mark price + funding + liquidation + insurance fund/ADL**. So step 1 is literally "build a spot exchange"; the derivatives layer sits on top.

0. The reference you already know (Anthony GG) — and its real repos

He builds a *spot* exchange, which is the correct foundation. Use it for the engine, then add the perp layer from section 3 onward.

Resource	Link
anthdm/crypto-exchange — "Build a crypto-exchange from scratch" series repo (Go)	https://github.com/anthdm/crypto-exchange
anthdm/rust-trading-engine — same idea, matching engine in Rust	https://github.com/anthdm/rust-trading-engine
EP1 — Matching Engine	https://www.youtube.com/watch?v=5r1wHkmb3HM
EP2 — Orderbooks	https://www.youtube.com/watch?v=SvitKOkJmm8
EP3 — Exchange API	https://www.youtube.com/watch?v=oE8TPzDIzLY
Matching market orders + settlement	https://www.youtube.com/watch?v=5oEOZjCvgiQ
Market maker algorithms	https://www.youtube.com/watch?v=1NF1-688sc8
Full playlist search (all episodes)	https://www.youtube.com/results?search_query=anthony+gg+crypto+exchange+from+scratch

1. Exchange fundamentals (read before writing code)

Resource	Why	Link
Jane Street — "How to Build an Exchange" (talk)	The single best free overview of exchange internals: order book, matching, fairness, sequencing. Watch first.	https://www.youtube.com/watch?v=b1e4t2k2KJY
Principles & Guidelines of Electronic Trading Systems Architecture	Architecture-level: gateways, sequencer, engine, market data, drop copy	https://www.youtube.com/watch?v=-L2XII0eXvY
How traders' orders get matched (matching algorithms)	Price-time vs pro-rata, FIFO, hidden/iceberg	https://www.youtube.com/watch?v=dLfnEKf1AFo
High-Throughput Stock Exchange — systems design	Sharding by symbol, sequencing, durability, failover	https://www.youtube.com/watch?v=dGYfpO3WJ1o
Inside a real HFT system — architecture	What latency budget actually means end-to-end	https://www.youtube.com/watch?v=iwRaNYa8yTw
Book: Larry Harris, <i>Trading and Exchanges: Market Microstructure for Practitioners</i>	The canonical textbook. Order types, market structure, liquidity, manipulation. Buy it.	ISBN 978-0195144703
NASDAQ TotalView-ITCH 5.0 spec (PDF)	Real production market-data protocol. Model your own book/feed messages on it.	https://www.nasdaqtrader.com/content/technicalsupport/specifications/dataproducts/itch50spec.pdf

Resource	Why	Link
NASDAQ TotalView product page	Context for the above	https://nasdaqtrader.com/trader.aspx?id=totalview2
LOBSTER — limit order book data structure docs	How an L3 book is represented and reconstructed	https://lobsterdata.com/info/DataStructure.php

2. Matching engine — designs + reference implementations to read

Design/theory

Resource	Link
LMAX Disruptor paper (single-threaded engine + ring buffer — the standard exchange-core pattern)	https://lmax-exchange.github.io/disruptor/disruptor.html
exchange-core wiki — architecture of a production-grade engine (event sourcing, risk engine, matching engine separation)	https://github.com/exchange-core/exchange-core/wiki

Code to read (pick 2-3, any language — the data structures are identical everywhere)

Repo	Lang	Link
exchange-core/exchange-core — most complete OSS exchange core (risk engine + matching + journaling)	Java	https://github.com/exchange-core/exchange-core
i25959341/orderbook — clean, readable price-time matching engine	Go	https://github.com/i25959341/orderbook
dyn4mik3/OrderBook — classic teaching implementation	Python	https://github.com/dyn4mik3/OrderBook
joaquinbejar/OrderBook-rs — thread-safe high-perf book	Rust	https://github.com/joaquinbejar/OrderBook-rs
brprojects/Limit-Order-Book — intrusive linked list + price-level map, 1.4M tx/s	C++	https://github.com/brprojects/Limit-Order-Book
AsthaMishra/matching-engine — ~100ns book ops, replays 104M real ops	Rust	https://github.com/AsthaMishra/matching-engine
KareemJandali/itchbook — ITCH 5.0 book reconstructor + matching engine	C++20	https://github.com/KareemJandali/itchbook
jordanbaucke/Limit-Order-Book — several alternative book designs compared	mixed	https://github.com/jordanbaucke/Limit-Order-Book
lanpishu6300/crypto-exchange — perp futures matching engine	—	https://github.com/lanpishu6300/crypto-exchange

Canonical data structure: price -> FIFO queue of orders in a sorted map (or array of price levels for a fixed tick grid), plus orderId -> node hash map for $O(1)$ cancel. Learn this once; it's the same in every language.

3. What a perpetual actually IS (the derivatives layer)

Resource	Why	Link
BitMEX — Perpetual Contracts Guide	The original perpetual swap. Invented the product; still the clearest primary spec.	https://www.bitmex.com/app/perpetualContractsGuide
BitMEX — Funding rate history & calculation	Premium index + interest rate → funding	https://www.bitmex.com/app/fundingRateHistory
BitMEX — Liquidation	Maintenance margin, liquidation price, liquidation engine	https://www.bitmex.com/app/liquidation
BitMEX — Auto-Deleveraging (ADL)	What happens when the insurance fund can't cover a bankruptcy. You must implement this.	https://www.bitmex.com/app/autoDeleveraging
Binance — funding rate / perp FAQ	Second reference implementation of the same math	https://www.binance.com/en/support/faq/detail/360033525031
Deribit — Insurance fund	Sizing, socialised loss	https://www.deribit.com/kb/deribit-insurance-fund
Deribit education blog	Options/futures/perp mechanics explained well	https://blog.deribit.com/education/
dYdX Academy — perps & funding rates (video)	5-minute intuition	https://www.youtube.com/watch?v=MMjR_LRhfK0
dYdX — how funding rates work (video)	https://www.youtube.com/watch?v=RcPr8bK0hY8	
Vertex — perpetual swaps in 2 minutes	https://www.youtube.com/watch?v=1D4HFy6VOro	
Paper: "Perpetual Futures Pricing" (arXiv 2310.11771)	Rigorous pricing/no-arbitrage theory of the funding mechanism	https://arxiv.org/abs/2310.11771
Paradigm — "The Cartoon Guide to Perps"	Best intuitive explainer of funding & vAMM-style perps	https://www.paradigm.xyz/2021/03/the-cartoon-guide-to-perps

Resource	Why	Link
Paradigm Research (index)	Deeper derivatives/market-design papers	https://research.paradigm.xyz/

The components you have to add on top of a spot engine

1. **Contract spec** — linear vs inverse, contract size, tick size, max leverage tiers.
2. **Index price** — median of N spot venues, outlier rejection, staleness guard.
3. **Mark price** — index + decayed funding basis (or impact-bid/ask EMA). Liquidations use mark, **never** last traded price.
4. **Margin accounting** — initial vs maintenance margin, isolated vs cross, unrealised PnL, position netting.
5. **Funding** — premium index → clamped funding rate → periodic cash transfer long ↔ short. Funding is peer-to-peer; the exchange is flat.
6. **Liquidation engine** — trigger at maintenance margin, partial/tiered liquidation, take over position at bankruptcy price.
7. **Insurance fund** → **ADL** waterfall when it's exhausted.
8. **Risk limits** — position size tiers lowering max leverage as size grows.

4. Production specs to copy (these are the best "requirements documents" that exist)

CEX API specs — read these as the spec for your own REST/WS API:

Venue	Link
Binance USD $\mathbb{S}$ -M Futures API docs	https://developers.binance.com/docs/derivatives/usds-margined-futures/general-info
Bybit V5 API (unified spot+perp)	https://bybit-exchange.github.io/docs/v5/intro
OKX V5 API	https://www.okx.com/docs-v5/en/
Bitfinex API docs	https://docs.bitfinex.com/docs
CCXT — unified model across 100+ venues (great for seeing what fields are universal)	https://docs.ccxt.com/

On-chain / DEX perp designs (excellent public write-ups of margin, funding, liquidation):

Protocol	What to steal	Link
Hyperliquid — margining	Cross/isolated margin, margin tiers	https://hyperliquid.gitbook.io/hyperliquid-docs/trading/margining
Hyperliquid — funding	Funding formula with clamps and interest component	https://hyperliquid.gitbook.io/hyperliquid-docs/trading/funding
Hyperliquid — liquidations	Backstop liquidator vault model	https://hyperliquid.gitbook.io/hyperliquid-docs/trading/liquidations
Hyperliquid — API	Clean perp API surface	https://hyperliquid.gitbook.io/hyperliquid-docs/for-developers/api
Hyperliquid Python SDK (read the types)	https://github.com/hyperliquid-dex/hyperliquid-python-sdk	
dYdX v4 — x/clob module source	A real on-chain CLOB perp exchange, fully open source: matching, funding, liquidations, ADL	https://github.com/dydxprotocol/v4-chain/tree/main/protocol/x/clob
dYdX v4 docs	https://docs.dydx.xyz/	
Drift Protocol docs	Hybrid AMM+orderbook, JIT auctions	https://docs.drift.trade/
GMX docs	Pool-based perps (no order book) — the alternative design	https://docs.gmx.io/
Synthetix docs	Synthetic perps, skew-based funding	https://docs.synthetix.io/
Perpetual Protocol v2 contracts (vAMM)	https://github.com/perpetual-protocol/perp-curie-contract	
Aevo docs	Off-chain orderbook, on-chain settlement	https://docs.aevo.xyz/
Paradex docs	StarkEx-based perp DEX	https://docs.paradex.trade/

5. Oracles / price feeds (you need an index price)

Resource	Link
Pyth — low-latency price feeds (crypto perp standard)	https://docs.pyth.network/price-feeds
Chainlink Data Feeds	https://docs.chain.link/data-feeds

Start simpler: pull spot books from 3–5 CEXes over WebSocket, drop stale/outlier feeds, take the median, EMA it. Write the outlier rules down — most perp exchange blowups are oracle blowups.

6. Risk management & clearing (the part hobby projects skip)

Resource	Why	Link
CME SPAN margin overview	How real clearinghouses margin portfolios	https://www.cmegroup.com/clearing/risk-management/span-overview.html
Campbell Harvey (Duke) — free finance/crypto course materials & papers	Rigorous background on derivatives and DeFi	https://people.duke.edu/~charvey/
OSQuant — quant research articles	Backtesting, funding-rate strategies, microstructure	https://osquant.com/

7. Suggested build order

1. **Order book + matching engine**, single symbol, in-memory, deterministic, single-threaded. Unit-test against a replay of real ITCH/L3 data.
2. **Event sourcing**: every inbound command appended to a log before the engine sees it; engine state = fold(log). Gives you crash recovery, replay, and audit for free. (See `exchange-core`.)
3. **Accounts + balances + spot settlement**. Integer/fixed-point math only — never floats for money.
4. **Gateway layer**: REST for orders, WebSocket for book/trade/user streams. Idempotent client order IDs.
5. **Index + mark price service** (separate process, feeds the engine).
6. **Margin & position engine**: initial/maintenance margin, unrealised PnL, cross vs isolated.
7. **Funding job**: premium index sampling → clamped rate → settlement every 1h/8h.
8. **Liquidation engine + insurance fund + ADL queue** (rank by profit × leverage).
9. **Risk limit tiers**, fee tiers, maker rebates, self-trade prevention.
10. **Market maker bot** to create liquidity for testing (Anthony GG's market-maker episodes cover this).
11. Chaos-test: oracle freeze, gap-down 30%, mass liquidation cascade, engine restart from log.

8. Hard-won rules

- **Fixed-point integers everywhere**. Price and size in ticks/lots. Floats in a matching engine = money leaks.
- **One thread per market**. Determinism beats parallelism; shard across symbols, not within one.
- **Sequence everything**. One monotonic sequence number per market, stamped before matching — it's your source of truth for replay and for clients detecting gaps.
- **Liquidate on mark price, not last price**. Otherwise a single wick liquidates your whole book.
- **The exchange must stay solvent, not fair**. Insurance fund → ADL is the backstop; design it before launch, not after.
- **Legal**: offering leveraged crypto derivatives is regulated in most jurisdictions. Build it as a learning project / testnet / simulated venue unless you have counsel.

9. Search shortcuts for more

- GitHub: <https://github.com/search?q=perpetual+futures+matching+engine&type=repositories&s=stars>
- GitHub: <https://github.com/search?q=limit+order+book+matching+engine&type=repositories&s=stars>
- YouTube: https://www.youtube.com/results?search_query=build+crypto+exchange+matching+engine+from+scratch